

Day 03: Spring Boot Fundamentals & Authentication Service

By the end of this session, learners will be able to:

- Understand backend system design using Spring Boot
- Implement a layered architecture for scalable services
- Design and integrate relational databases using PostgreSQL
- Develop secure authentication APIs using JWT
- Apply Role-Based Access Control (RBAC)
- Implement data validation, encryption, and error handling
- Integrate backend APIs with frontend systems

Backend System Perspective

Backend systems are responsible for:

- Business logic execution
- Data processing and persistence
- Security enforcement
- API exposure

Academic Insight

A well-designed backend ensures data **integrity**, **scalability**, and **security**, forming the backbone of any distributed system.

Spring Boot Architecture

Spring Boot is built on the Spring Framework, enabling rapid development of production-ready applications.

Core Features:

- Auto-configuration
- Embedded server (Tomcat)
- Dependency Injection (IoC Container)
- REST API support

Internal Flow (Abstract)

Request → Controller → Service → Repository → Database → Response

Layered Architecture

Layered architecture enforces separation of concerns and improves maintainability.

Layers Defined

Module	Purpose
Controller	Handles HTTP requests
Service	Business logic
Repository	Database interaction
Entity	Data Representation

Advantages:

- Modular codebase
- Easy testing

- Improved scalability

Theoretical Principle:

Layered architecture aligns with SOLID principles, particularly Single Responsibility Principle (SRP).

PostgreSQL Integration

PostgreSQL is used for structured data storage.

Integration Components

- JDBC Driver
- JPA (Java Persistence API)
- Hibernate ORM

Configuration Concept

- Database URL
- Username/Password
- ORM mapping

Example:

```
@Entity
class User {
    @Id
    private Long id;
    private String email;
}
```

User Entity Design

Attributes – id, name, email, password, role

Design Considerations

- Unique email constraint
- Password must be encrypted
- Role defines access level

Insight:

Entity design must reflect real-world constraints and relationships to ensure data consistency.

DTOs & Validation

DTO (Data Transfer Object)

DTOs are used to:

- Transfer data between layers
- Prevent exposure of internal entities

Validation

Validation ensures correctness of input data.

Common Constraints

- NotNull
- Email format
- Password length

Example

```
class RegisterDTO {  
    @Email  
    private String email;  
  
    @Size(min = 6)  
    private String password;  
}
```

Academic Insight:

Validation enforces data integrity at system boundaries, reducing runtime errors.

REST APIs for Authentication

REST APIs follow stateless communication principles.

Core Endpoints

Endpoint	Method	Purpose
/auth/register	POST	User registration
/auth/login	POST	User login
/auth/logout	GET	User logout
/auth/forgot-password	POST	Forgot Password
/auth/reset-password	POST	Reset Password

Example Flow: Client → API → Service → DB → Response

JWT Authentication

JWT (JSON Web Token) provides stateless authentication.

Flow

- User submits credentials
- Server validates
- Token generated
- Token sent in response
- Token used in subsequent requests

Conceptual Structure

- Header
- Payload
- Signature

Security Insight:

JWT eliminates server-side session storage, enabling scalable distributed systems.

Role-Based Access Control (RBAC)

RBAC restricts system access based on roles.

Roles Defined: USER, THEATRE_OWNER, ADMIN

Access Control Example

Endpoint	User	Theatre Owner	Admin
Book Tickets	Yes	No	No
Manage Shows	No	Yes	Yes
Manage Users	No	No	Yes

Implementation Concept

- Role stored in database
- Checked during request processing

Theoretical Principle:

RBAC enforces least privilege principle, enhancing system security.

BCrypt Password Encryption

Purpose

- Secure password storage
- Prevent plaintext exposure

Concept: Password → Hashed → Stored

Key Property: One-way hashing (non-reversible)

Security Insight: Encryption protects against data breaches and unauthorized access.

API Integration with Frontend

Frontend communicates via HTTP APIs.

Example

- Login request sent from React frontend
- Response contains JWT

Integration Flow: Frontend → API → Auth Service → Response

Design Insight: Backend APIs must be consistent, predictable, and well-documented.

Error Handling & Validation

Types of Errors

- Validation errors
- Authentication failures
- Server errors

Standard Response Format

```
{  
  "message": "Invalid credentials",  
  "status": 401  
}
```

Best Practices

- Use meaningful messages
- Maintain consistent structure
- Avoid exposing sensitive data

Academic Insight: Error handling improves system robustness and user experience.

Learning Outcomes

Students will be able to:

- Design backend services using layered architecture
- Implement secure authentication APIs
- Apply RBAC for access control
- Integrate database with backend systems
- Handle validation and errors effectively

Deliverables of Day 03

- Fully functional Authentication Service
- User registration & login APIs
- JWT-based authentication
- Role-based access control implemented
- PostgreSQL integration completed
- Error handling and validation applied

Viva Questions

1. What is Spring Boot and how does it simplify backend development?
2. Explain layered architecture in backend systems.
3. What is the role of JPA and Hibernate?
4. What is an Entity and how is it mapped to a database?
5. Why are DTOs used instead of entities?
6. What is validation and why is it important?
7. Explain REST API principles.
8. What is JWT and how does it work?
9. What are the advantages of stateless authentication?
10. Explain Role-Based Access Control with examples.
11. What is BCrypt and why is it used?
12. How does backend integrate with frontend?
13. What are common types of API errors?
14. Why is error handling important in backend systems?
15. What is the difference between authentication and authorization?
16. What are best practices for API design?
17. How does database design impact backend performance?
18. What are the security considerations in authentication systems?